

COMP9820 Software Project Management

Agile Software Architecture and Design for Web Applications

Dr. Basem Suleiman

School of Computer Science and Engineering (CSE)



Acknowledgement of Country

"I acknowledge the Bidjigal as the Traditional Custodians of the land we are meeting on today. I pay my respects to Elders past and present and recognise the enduring connection of Aboriginal and Torres Strait Islander peoples to Country. I invite us all to reflect on our shared responsibility to honour and respect this land and its stories."

Outline

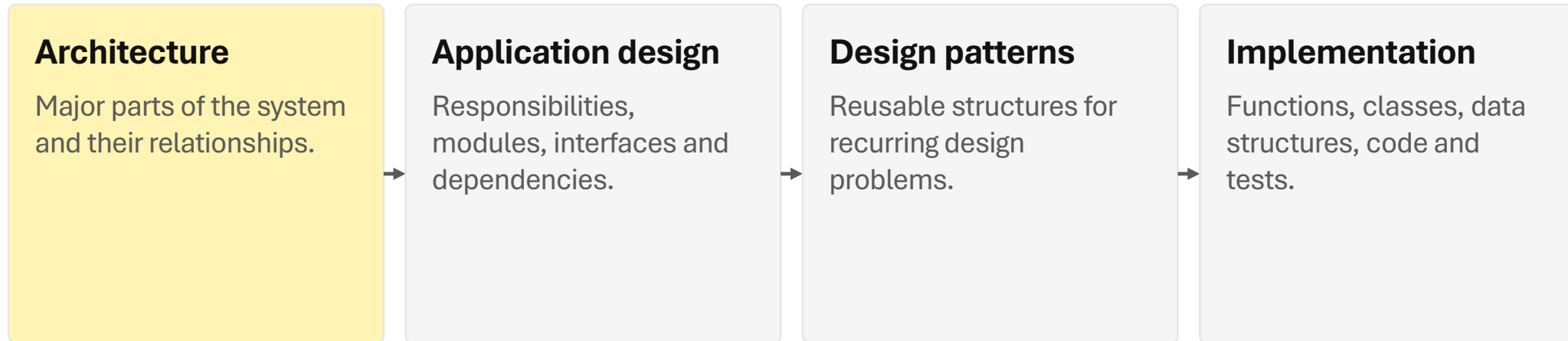
- **Key Principles: Software Architecture and Design**
- **How engineers think about and communicate design**
- **Agile design: connecting principles to architecture**
- **Web application components: front end, API, back end and data**
- **Design principles for non-functional requirements / qualities**
- **Typical Software Architectural Patterns**
- **Design decisions within Scrum and CI/CD**
- **Architecture and design choices and its implications**
- **Project Work – Sprint 1 Assessment**

Software Architecture and Design

Core principles

Software architecture, design and patterns

Different levels of the same reasoning process



The boundary is not rigid.

Architecture focuses on the decisions that are expensive or risky to change later. Design continues inside those architectural boundaries.

Design operates at different levels

Architecture

Major parts and communication

Application structure

Responsibilities and boundaries

Component design

Interfaces and dependencies

Implementation

Functions, classes and tests

Move between levels as needed. Do not start with code-level detail when the system boundary is unclear.

How software engineers think about design

What problem are we solving?

What must change later?

What belongs together?

What should stay separate?

How will we test it?

What could fail?

Architecture diagrams support communication

1

Context

Who uses the system?

2

Containers

What applications and data stores exist?

3

Components

What responsibilities sit inside an application?

4

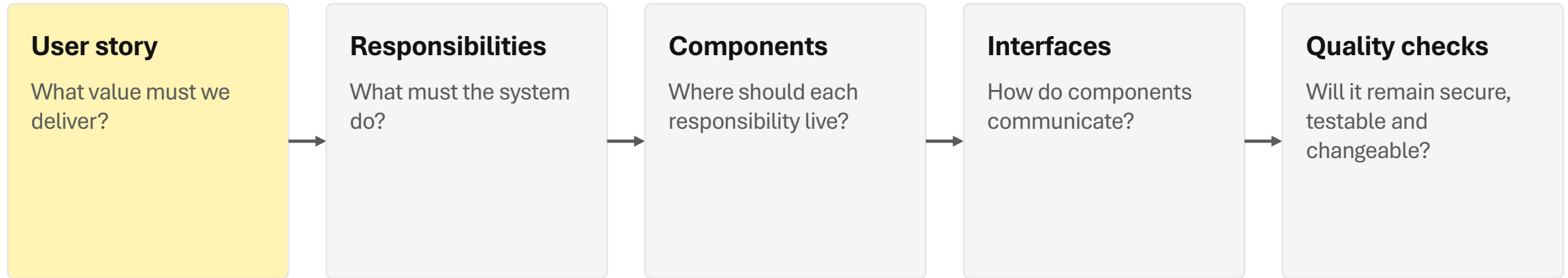
Flow

How does a request move through the system?

Tools: whiteboard • diagrams.net • Mermaid • PlantUML • C4-based tools

How software engineers think about design

Start from a need, then assign responsibilities and boundaries



Example user story

As a group member, I want to record an expense so that the group knows who paid and what everyone owes.

FROM AGILE TO DESIGN – Class Activity



What guides Agile teams in creating software architecture and design?

How does the Agile methodology approach software architecture and design?

Agile Manifesto –Values

- “We are uncovering **better ways of developing software** by doing it and helping others do it. Through this work we have come to **value:**”
 - **Individuals and interactions** over processes and tools
 - **Working software** over comprehensive documentation
 - **Customer collaboration** over contract negotiation
 - **Responding to change** over following a plan
- The items on the left are **more valued** than those at the right

Agile Manifesto: <http://agilemanifesto.org/>

© 2001, the above authors. This declaration may be freely copied in any form, but only in its entirety through this notice.

Agile Principles

1. Our highest priority is to satisfy the customer through early and continuous delivery of valuable software .	5. Build projects around motivated individuals . Give them the environment and support they need and trust them to get the job done.	9. Continuous attention to technical excellence and good design enhances agility.
2. Welcome changing requirements , even late in development. Agile processes harness change for the customer's competitive advantage.	6. The most efficient and effective method of conveying information to and within a development team is face-to-face conversation .	10. Simplicity --the art of maximizing the amount of work not done--is essential.
3. Deliver working software frequently , from a couple of weeks to a couple of months, with a preference to the shorter timescale .	7. Working software is the primary measure of progress .	11. The best architectures, requirements, and designs emerge from self-organizing teams .
4. Business people and developers must work together daily throughout the project.	8. Agile processes promote sustainable development . The sponsors, developers, and users should be able to maintain a constant pace indefinitely.	12. At regular intervals, the team reflects on how to become more effective , then tunes and adjusts its behavior accordingly.

Agile principles create technical design responsibilities

Agile values and design decisions

Working software

Design enough structure to deliver a usable increment

Responding to change

Keep responsibilities and dependencies easy to change

Technical excellence

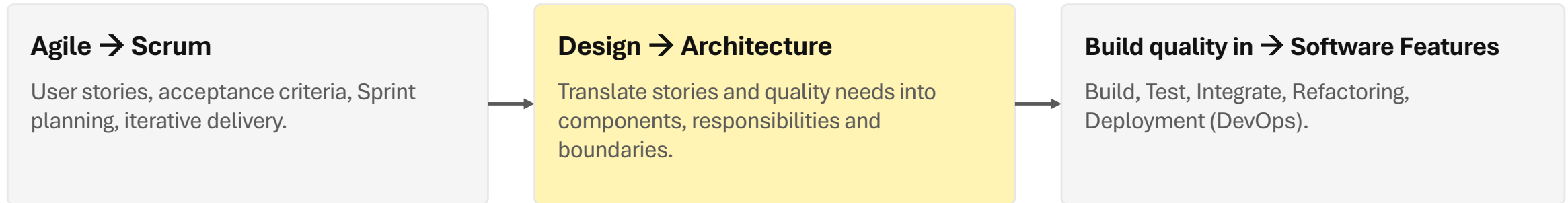
Use testing, refactoring and clear interfaces to sustain speed

Customer collaboration

Use feedback to validate both behaviour and priorities

The technical choices behind an increment affect how easily the team can respond to change

From Agile to Software Design & Architecture



Turning Agile planning into technical design decisions, and software features to be delivered and deployed

Agile principles make the bridge explicit:

- Continuous attention to technical excellence and good design enhances agility.
- Simplicity helps teams avoid unnecessary work.
- Architectures and designs can emerge through collaboration and feedback.

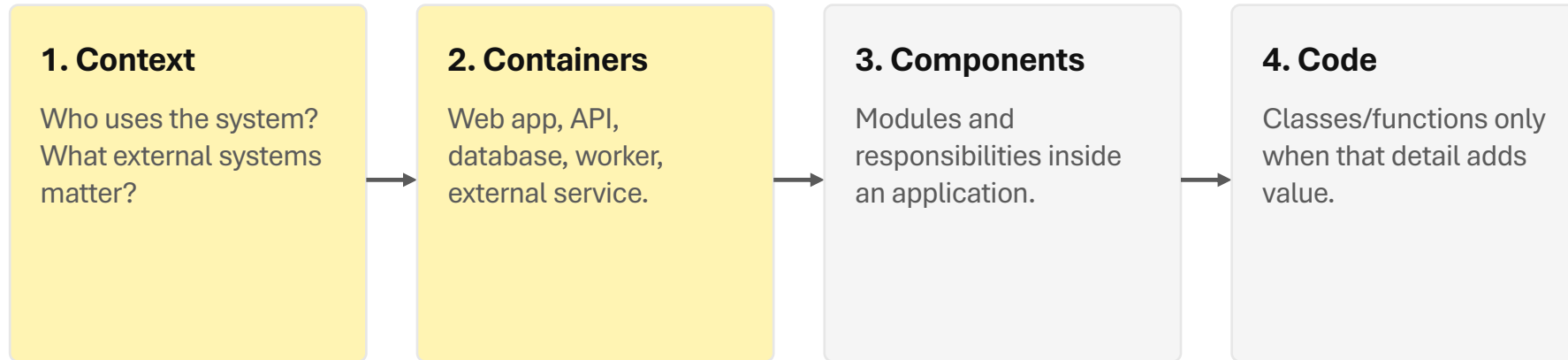
What you should be able to learn

- Identify the main components of a web application
- Explain the difference between architecture, design principles and design patterns
- Draw a simple architecture or request-flow diagram
- Use separation of concerns, cohesion and coupling to discuss a design
- Relate design decisions to maintainability, testability and extensibility
- Explain how design evolves within Scrum and CI/CD

How to draw architecture

Draw enough to answer a question, not every detail

A useful zoom model



Tools

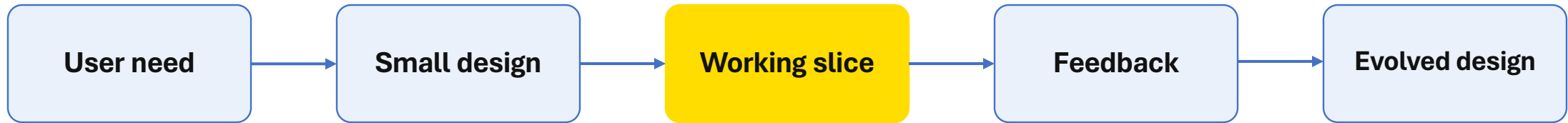
Whiteboard
diagrams.net
Lucidchart
Miro
Mermaid
PlantUML
Structurizr

For most student web projects, a context diagram + a container diagram is usually enough to start.

Design in an Agile development

From a user story to Web application components

Agile design is iterative and incremental



Design decisions become more reliable when the team tests them through working increments.

Good design is not a document handed to developers. It is a set of decisions that makes change safer, clearer and cheaper.

A user story becomes a set of responsibilities

**As a registered user,
I want to submit a request
and track its status,
so that I know what is happening.**

Present the form

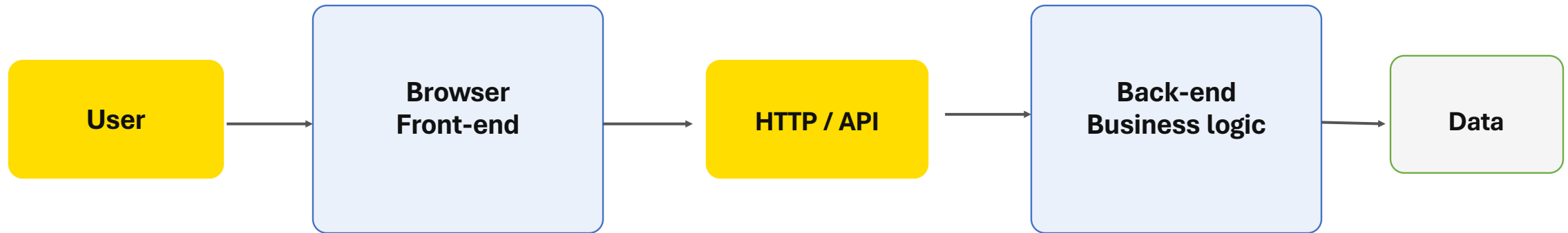
Validate the request

Apply business rules

Store and retrieve status

Basic web application architecture

A browser communicates with server-side software through HTTP



The boundary is also a responsibility boundary: the server must enforce rules and protect data.

Class Activity: Classify front-end and back-end responsibilities

- Validates requests and permissions
- Applies business rules
- Reads and writes persistent data
- Integrates external services
- Returns useful responses and errors

- Displays information and accepts input
- Manages interaction and client-side state
- Calls the back end through an API
- Provides immediate feedback
- Handles loading, error and empty states

Front end and back end have different responsibilities

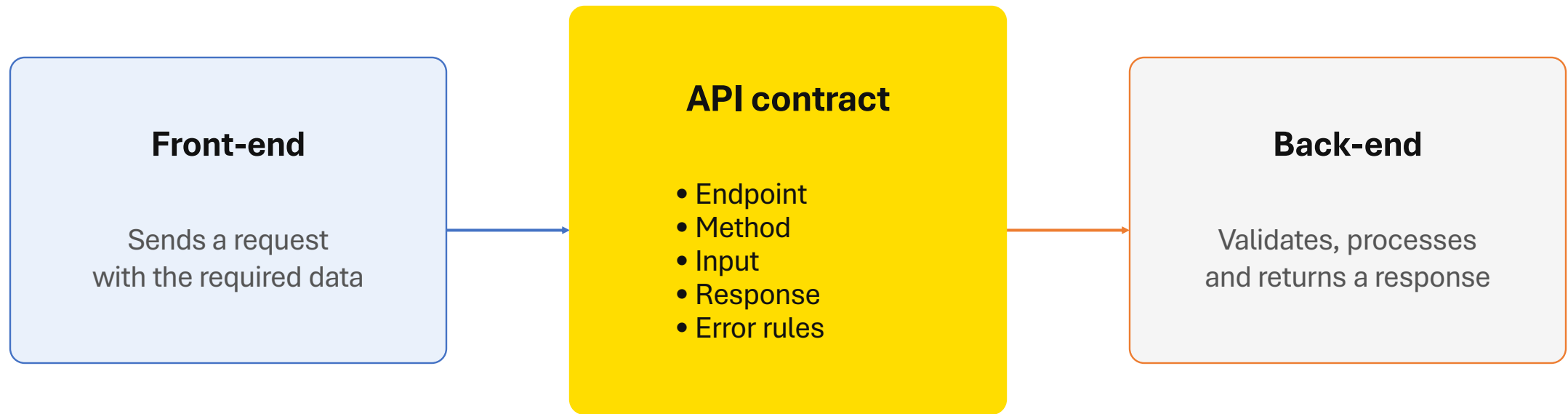
Front-end

- Displays information and accepts input
- Manages interaction and client-side state
- Calls the back end through an API
- Provides immediate feedback
- Handles loading, error and empty states

Back-end

- Validates requests and permissions
- Applies business rules
- Reads and writes persistent data
- Integrates external services
- Returns useful responses and errors

The API is a contract between components



Good structure can help to make better changes

Core design principles

Key principles worth considering for every Sprint

Separate concerns

Different responsibilities should not be tangled together.

Encapsulation

Hide internal details behind a clear interface.

High cohesion

Keep closely related behaviour together.

Low coupling

Minimise how much one part must know about another.

Simple dependencies

Make dependency direction visible and predictable.

Design for change

Isolate likely sources of change.

YAGNI* / simplicity

Do not build complexity for imagined future needs.

* You aren't gonna need it

Refactor continuously

Improve the design as understanding grows.

Separation of concerns

One feature can cross layers without mixing responsibilities

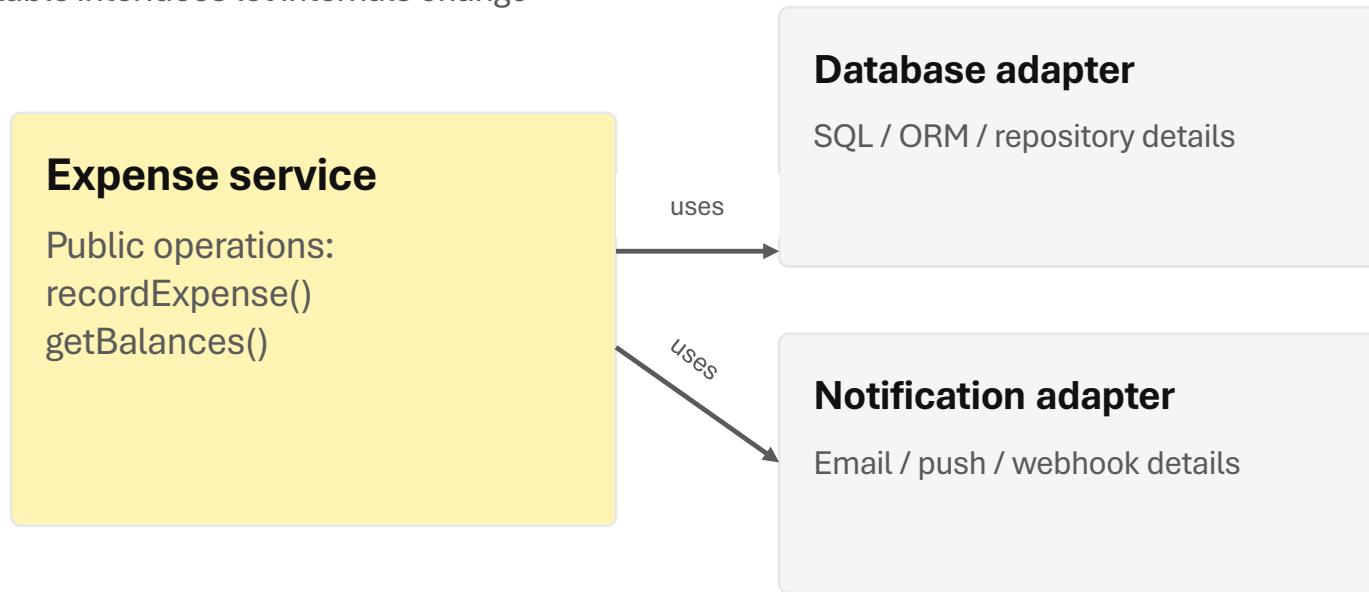


Same user story, distinct responsibilities

A change to the screen layout should not require rewriting persistence. A change to the database should not change the user story.

Encapsulation and dependency direction

Stable interfaces let internals change



Why it helps

Tests can replace adapters with fakes.

Infrastructure can change without rewriting business rules.

The goal is not “more layers”. The goal is explicit boundaries and fewer surprises.

Separation of concerns and encapsulation

Tangled responsibility

One module

- renders the form
- validates permissions
- calculates fees
- writes to the database
- sends email

Separated responsibility

Form / view
Business service
Data access
Notification adapter

Cohesion and coupling

Keep related things together; keep unnecessary dependencies apart

High cohesion + low coupling

Expense module

- create expense
- validate split
- calculate balances

Notification module

- email
- push notification

Low cohesion + high coupling

UtilityManager

- expense calculations
- email
- database queries
- authentication
- report formatting
- UI helpers

Design question: *“If I change this responsibility, how many unrelated places must I understand and modify?”*

Design for change – Class Activity

Architecture 1

12 services
3 message brokers
custom plugin platform
global multi-region replication
“because we may need it later”

Architecture 2

One deployable web app
clear modules
well-defined API boundaries
automated tests
room to extract a service later if evidence appears

Which Architecture would you choose? Why?

Simplicity, YAGNI and design for change

Keep today simple; protect the places that are likely to change

Do not build this on Sprint 1...

Speculative architecture

- 12 services
- 3 message brokers
- custom plugin platform
- global multi-region replication
- “because we may need it later”

Prefer this...

Simple, modular starting point

- One deployable web app
- clear modules
- well-defined API boundaries
- automated tests
- room to extract a service later if evidence appears

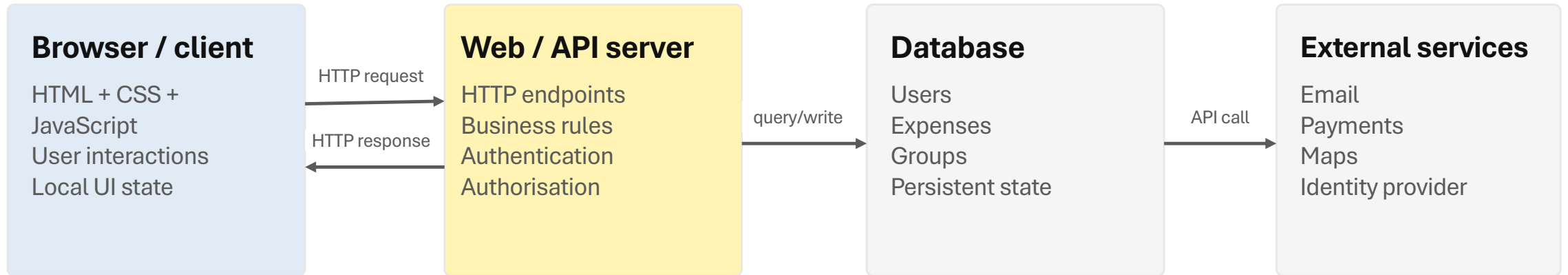
Design for change means isolating known instability, not predicting every future feature.

Architecture of Web applications

A browser click becomes a chain of responsibilities across the front end, network, back end, data and external services.

The simplest useful web architecture

Client → server → data



The browser is not “the application”. It is one part of a larger system.

Front-end: what belongs in the browser?

Presentation and interaction, not trusted business authority

- Render the interface and data
- Collect user input
- Manage UI state
- Provide immediate validation feedback
- Call back-end APIs
- Handle loading, errors and empty states
- Support accessibility and responsive layout

Example: “Record expense” form

The browser can check that amount looks numeric and show errors instantly.

But the server still validates

A user can bypass JavaScript or call the API directly. Business rules and authorisation must be enforced server-side.

Back-end: what belongs on the server?

Business behaviour, security enforcement, persistence and integration

API boundary

Receive HTTP requests; return structured responses.

Validation

Check required data and domain rules.

Authentication

Establish who the user is.

Authorisation

Decide what that user is allowed to do.

Business logic

Calculate splits, balances and other domain behaviour.

Persistence

Read/write durable data through a data access layer.

Integration

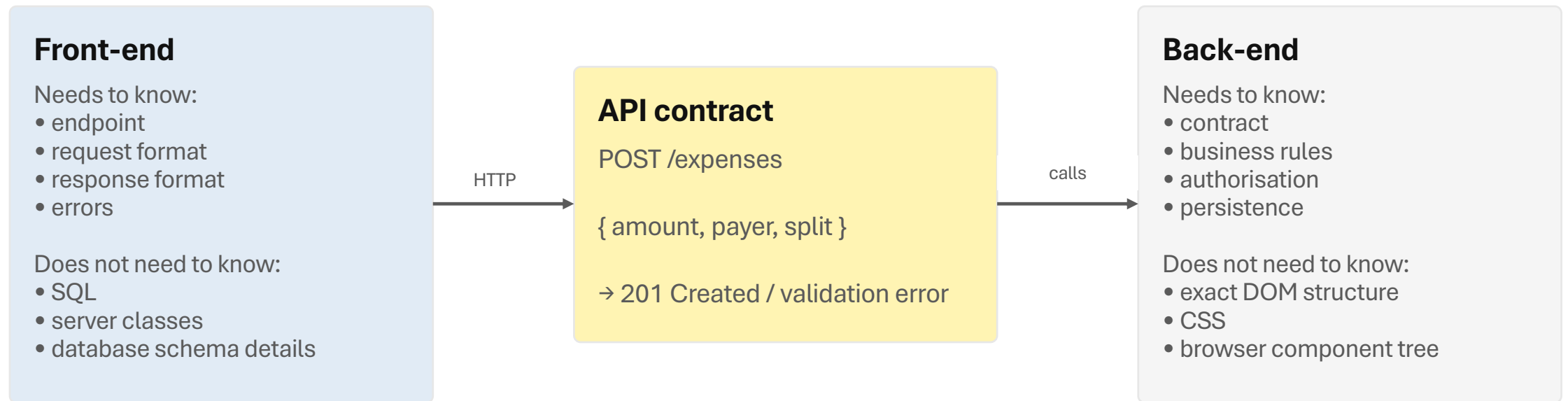
Call external services through controlled adapters.

Observability

Log important events and failures.

The API is a contract

A boundary lets front end and back end evolve with less coupling

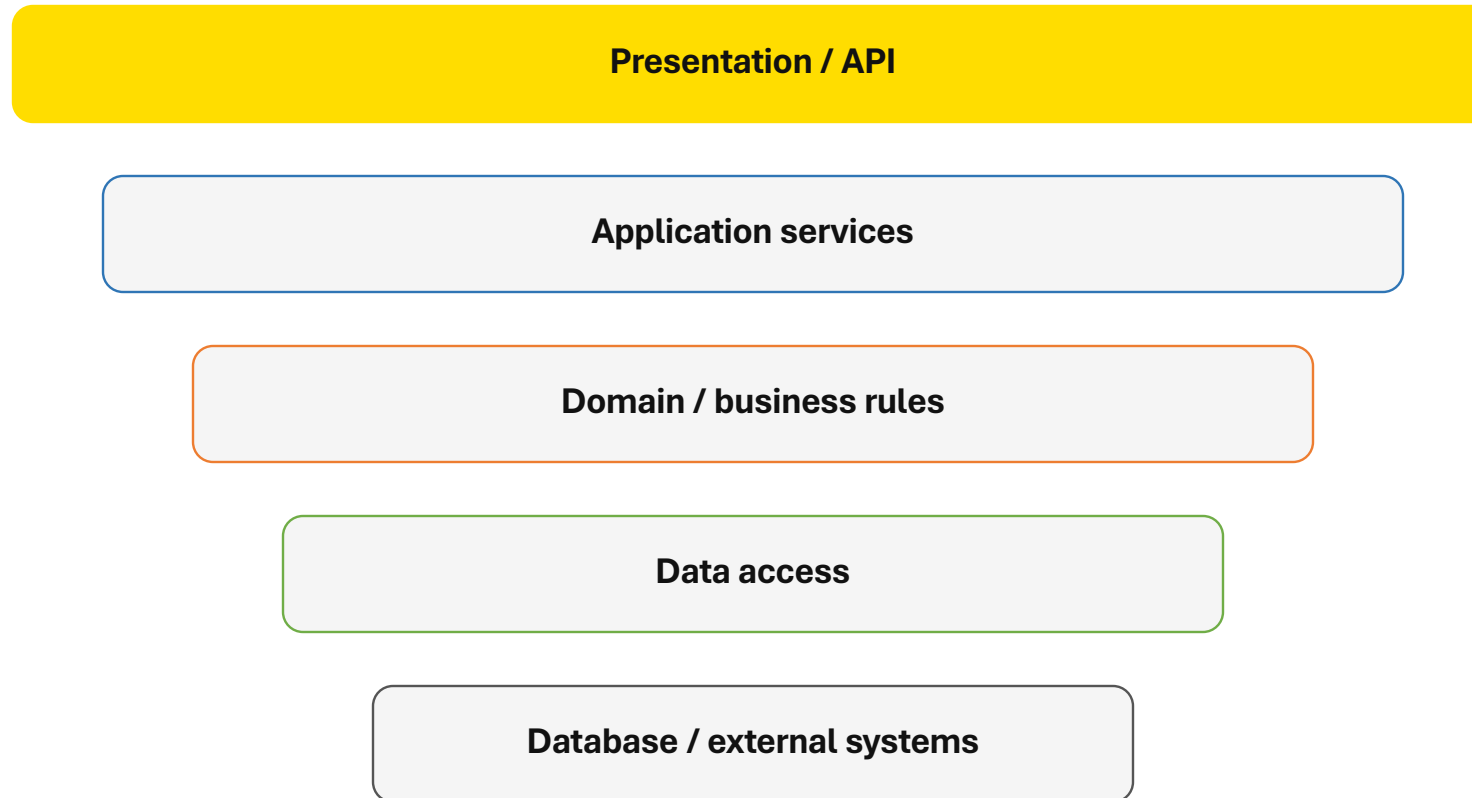


Good contracts reduce coupling. They do not eliminate coordination.

Typical Architectural Patterns

Architectural Patterns

Layered architecture

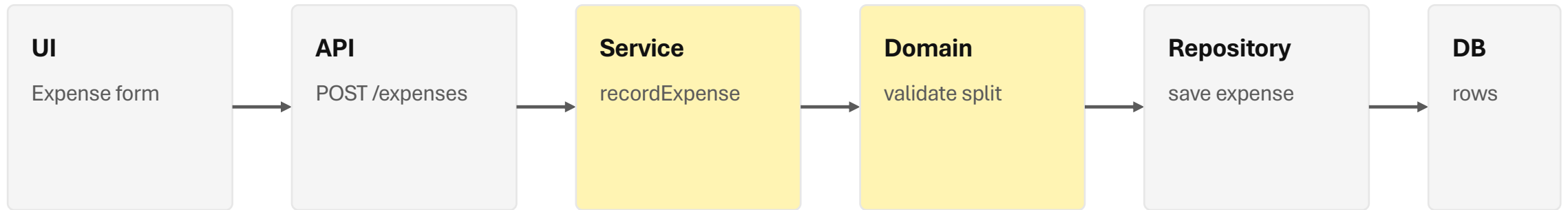


A request usually moves downward, while a response moves back upward.

Each layer has a primary responsibility

Layered architecture: one user story end to end

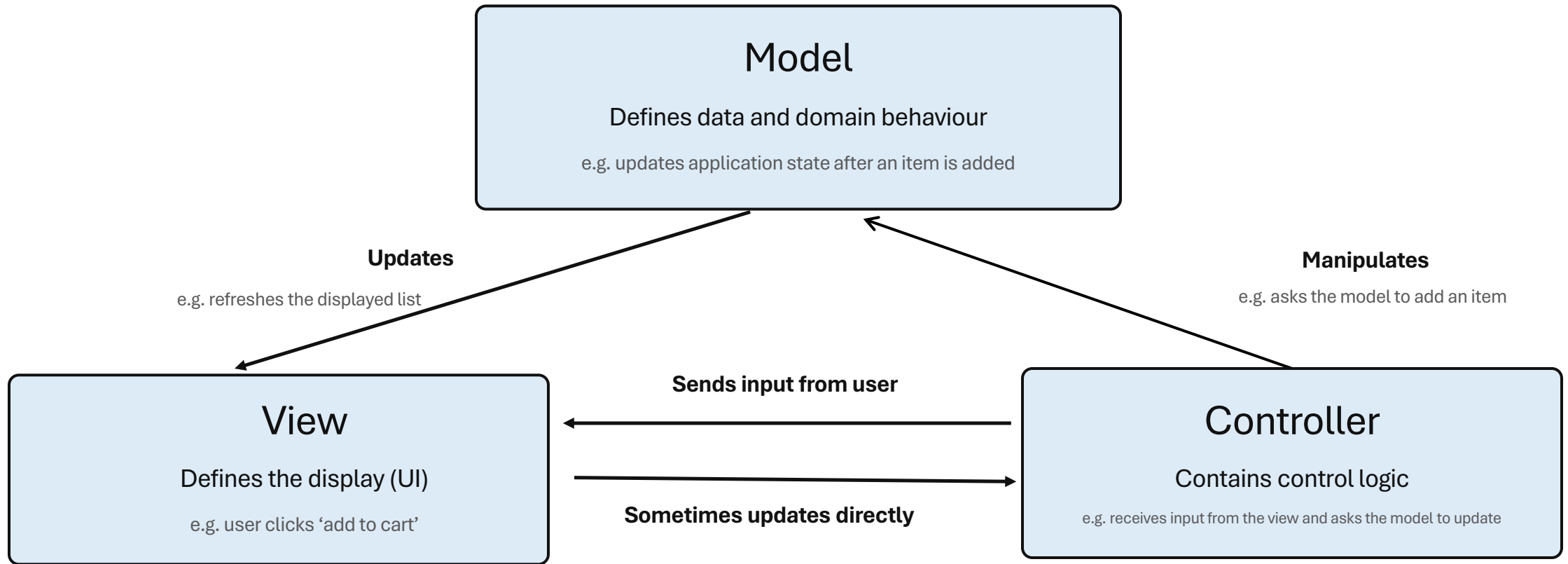
“Record an expense” as a vertical slice



Tests can exist at several seams:

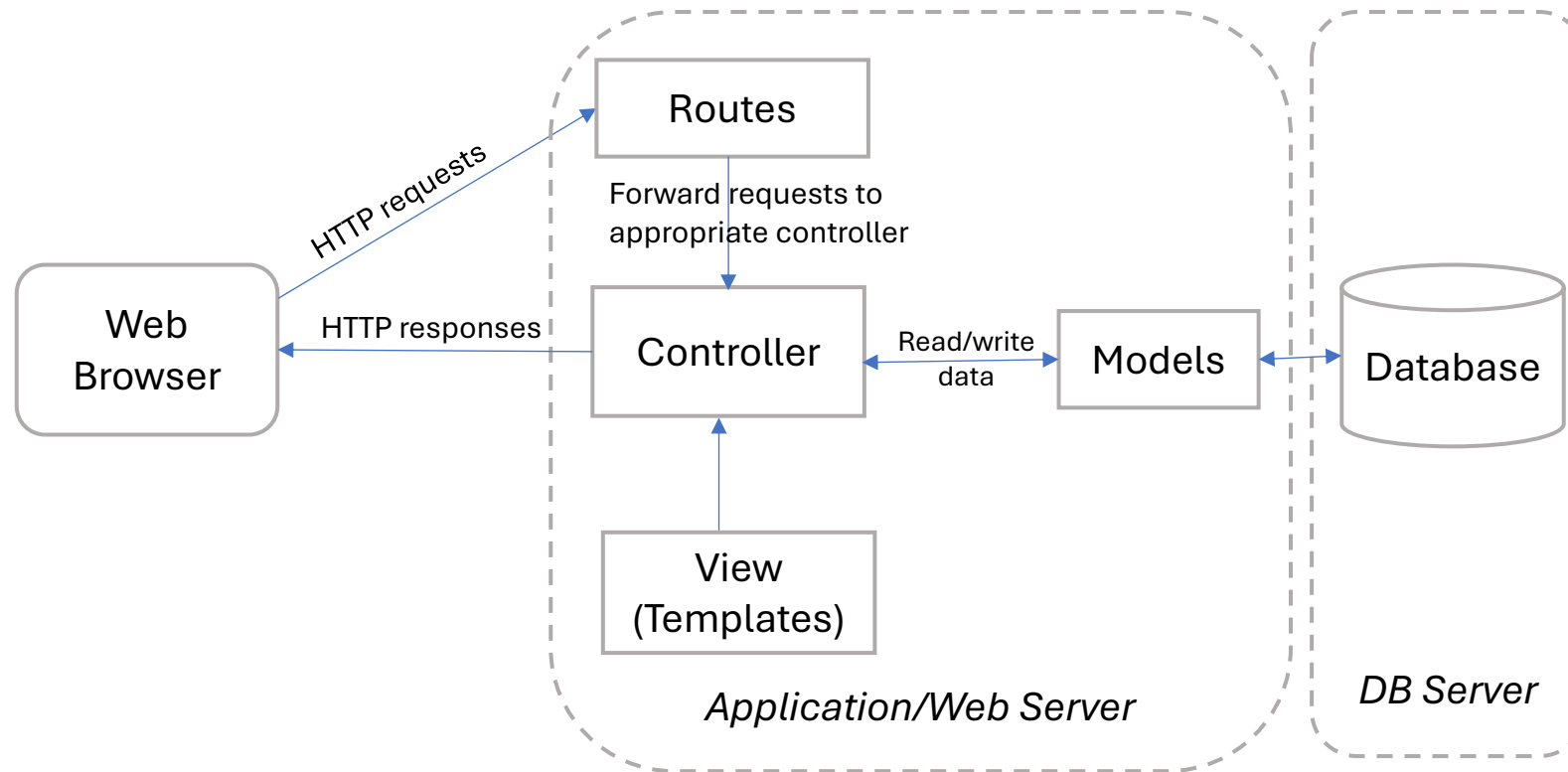
- Domain unit tests for split calculations
- Service tests with a fake repository
- API integration tests
- Small browser/end-to-end tests for critical user journeys

Model-view-controller (MVC)

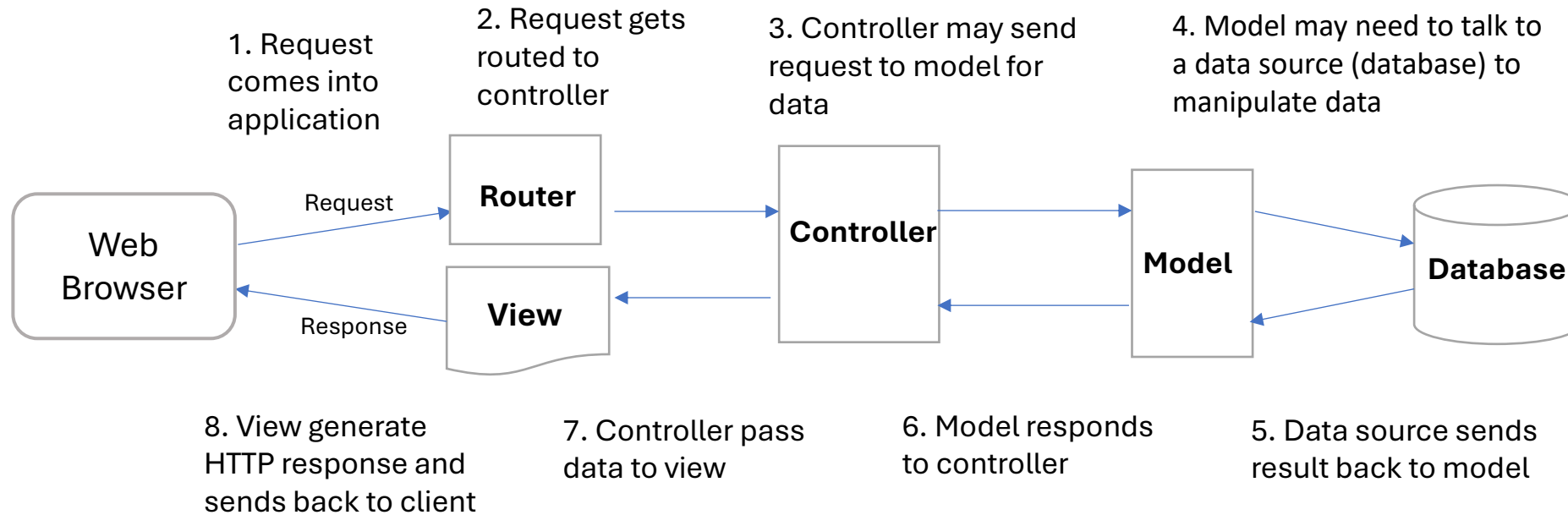


A pattern for separating data, presentation and request coordination

Model-view-controller (MVC) – Web Application

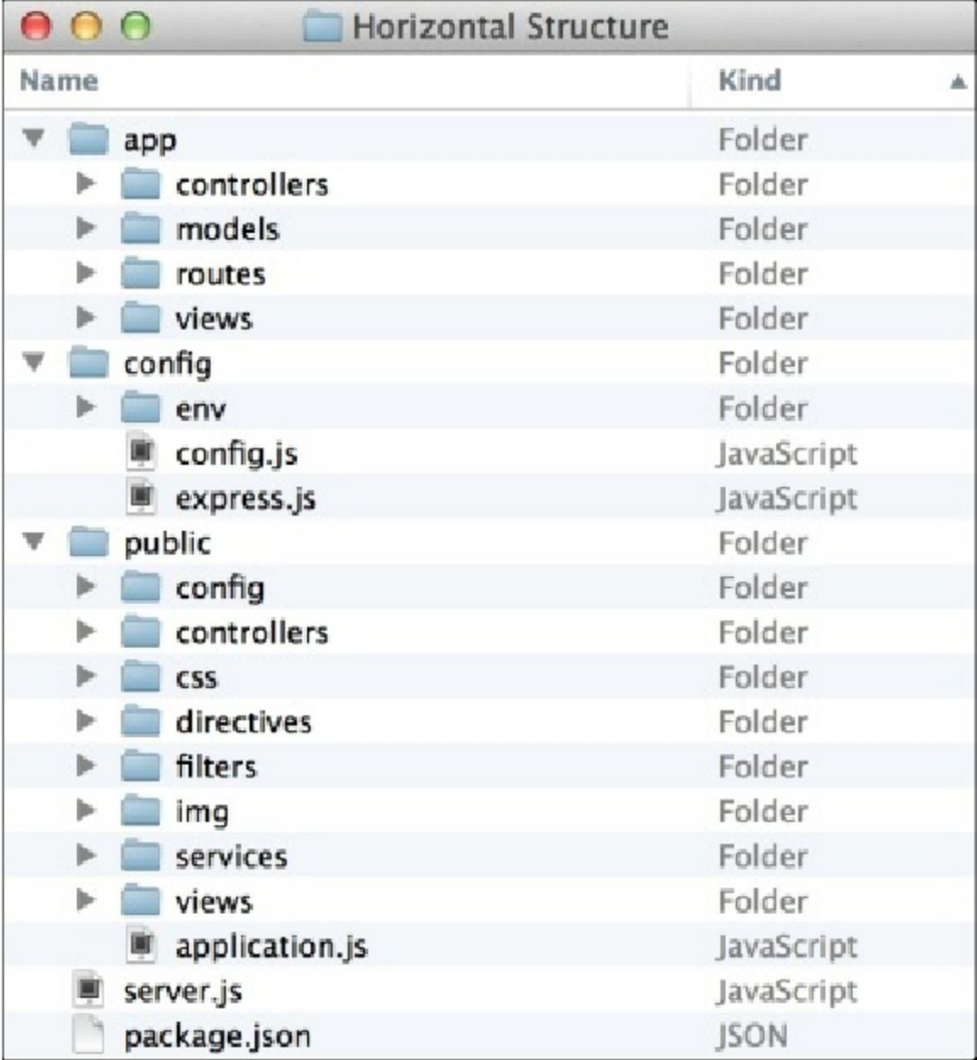


Model-view-controller (MVC) – Web Application



- Database related code should be put in model layer
- Controller should not have knowledge about the actual database
- Modularity allows easy switching between technologies
 - e.g. different view templates, different database management systems

Model-view-controller (MVC) – Web Application

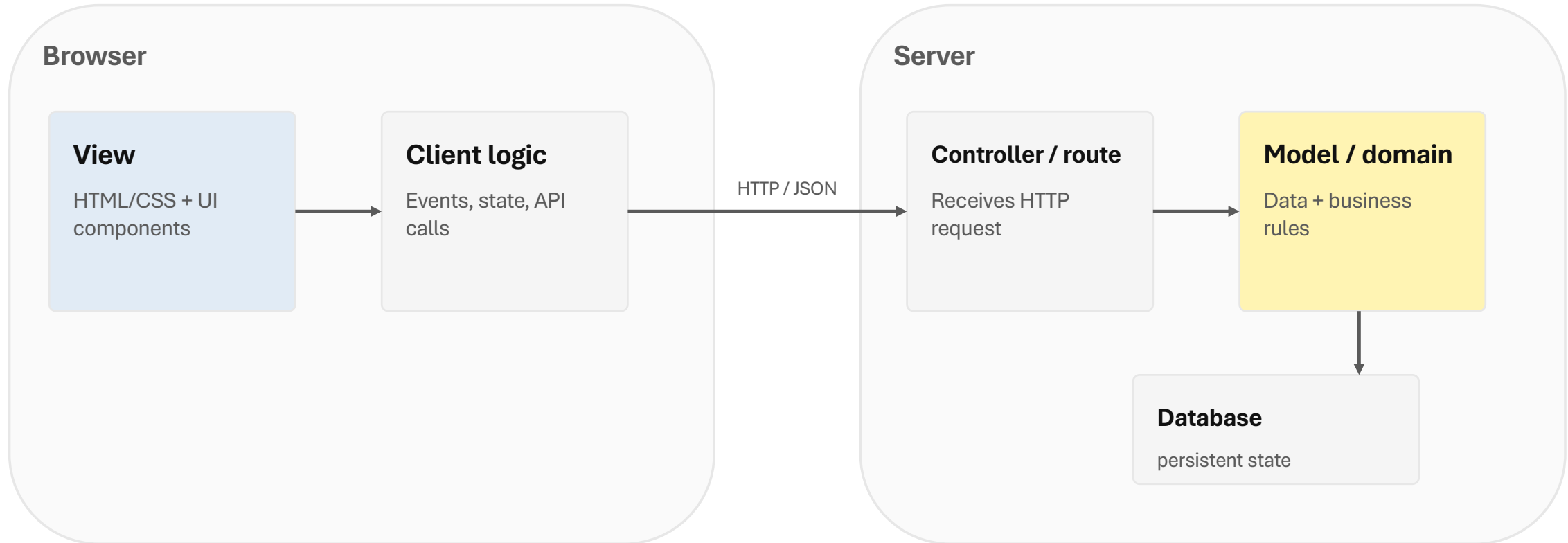


A screenshot of a file explorer window titled "Horizontal Structure" showing a project directory layout. The window has a table with two columns: "Name" and "Kind". The directory structure is as follows:

Name	Kind
▼ app	Folder
▶ controllers	Folder
▶ models	Folder
▶ routes	Folder
▶ views	Folder
▼ config	Folder
▶ env	Folder
config.js	JavaScript
express.js	JavaScript
▼ public	Folder
▶ config	Folder
▶ controllers	Folder
▶ css	Folder
▶ directives	Folder
▶ filters	Folder
▶ img	Folder
▶ services	Folder
▶ views	Folder
application.js	JavaScript
server.js	JavaScript
package.json	JSON

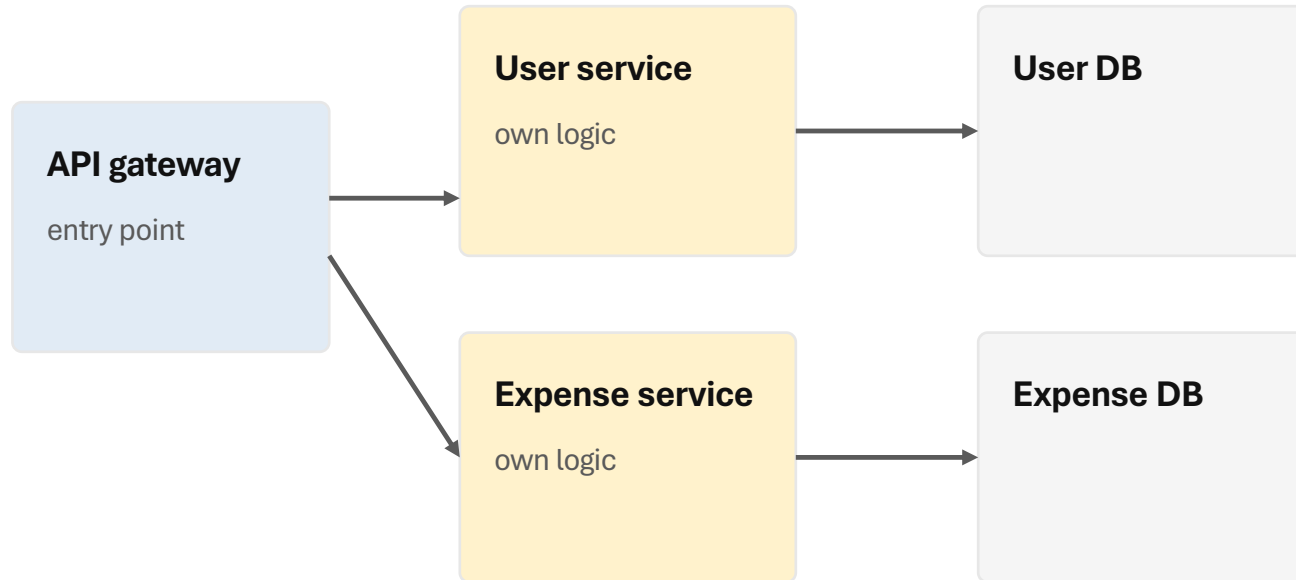
MVC in modern Web app

The responsibilities may be split across browser and server



Python example: Django describes its approach as Model–Template–View (MTV), similar in style to MVC.

Microservices Architecture



Potential benefits

Independent deployment and scaling
Clear service ownership
Fault isolation when designed well

New complexity

Network failures + latency
Distributed tracing
Data consistency
More CI/CD + operations
Harder cross-service refactoring

- Do not choose microservices because they “sound scalable”. Choose them when the benefits justify the operational complexity.
- Independent deployment can help, but distribution has a price

Making architecture choices and trade-offs

Architecture choices involve trade-offs

Question	Why it matters
What is changing?	Separate likely points of variation
How large is the team?	Avoid operational overhead the team cannot support
What must scale?	Scale the relevant component or workload
How will we deploy?	Keep the architecture compatible with CI/CD
How will we test failures?	Prefer boundaries the team can observe and verify

Architecture also supports non-functional properties

Performance

Where might time or resources be spent?

Scalability

What grows: users, requests, data or integrations?

Reliability

What happens when a dependency fails?

Security

Who can perform this action? How is data protected?

Others

Based on the application domain and stakeholders' needs

What “good design” buys an Agile team

Quality attributes are not decorations added at the end

Maintainability

Change code without widespread rework.

Understandability

A teammate can locate and reason about the right place to change.

Testability

Behaviour can be verified automatically and in isolation.

Modularity

Work can be divided into meaningful units with clear ownership.

Extensibility

New behaviour can be added without rewriting unrelated code.

Deployability

Changes can move through CI/CD with predictable impact.

These qualities reinforce each other: Clear modules are easier to understand, test, refactor and deploy.

Design decisions are part
of the sprint

A vertical slice connects design to delivery



Each sprint can deliver a small end-to-end capability instead of a disconnected technical layer.

Sprint Design and Development – Class Activity

Component design and development

Sprint 1

database components

Sprint 2

Back-end design and development

Sprint 3

Front-end design and development

Vertical Slice Design and Development

Story A

UI → API → rule → DB → test → CI

Story B

UI → API → rule → DB → test → CI

Story C

UI → API → rule → DB → test → CI

As Agile Teams, which design and development approach would you follow? Why?

Vertical slices: design for incremental delivery

Avoid component-only planning

Sprint 1

database only

Sprint 2

back end only

Sprint 3

front end only

Prefer vertical increments

Story A

UI → API → rule → DB → test → CI

Story B

UI → API → rule → DB → test → CI

Story C

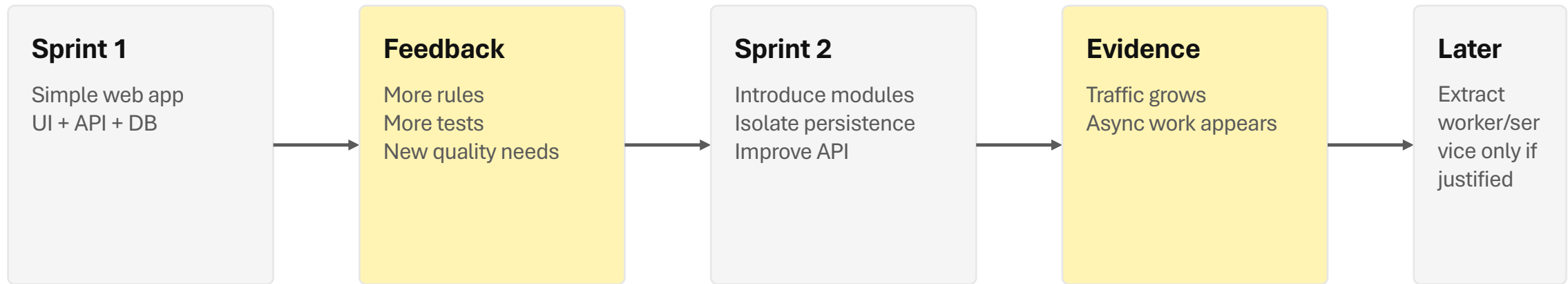
UI → API → rule → DB → test → CI

Each sprint can deliver a small end-to-end capability instead of a disconnected technical layer.

Why? Every Sprint produces integrated, testable behaviour and exposes architecture problems early.

A story should become a thin, working path through the architecture

Emergent architecture in Scrum



Emergent ≠ accidental

Architecture still needs intentional decisions, tests, refactoring and team communication. The difference is that decisions are revisited as evidence grows.

Design enough now, learn from the increment, then improve the structure

Applying the ideas to your COMP9820 project

- | | | |
|---|------------------------------|---|
| 1 | Before implementation | Agree on a simple system boundary and initial architecture |
| 2 | During the sprint | Keep design decisions visible in issues, merge requests and documentation |
| 3 | Before merging | Check responsibility boundaries, tests and integration |
| 4 | During review | Demonstrate working behaviour and collect stakeholder feedback |
| 5 | During retrospective | Record design or process improvements for the next sprint |

Key takeaways (1)

Architecture should make the next change easier, not merely look sophisticated

- Architecture organises the major parts of a system.
- Design principles help teams control responsibility and dependencies.
- Web applications separate front-end, API, back-end and data concerns.
- MVC and layered architecture are useful patterns, not universal rules.
- **Agile architecture evolves through increments, feedback and refactoring.**
- Good design makes software easier to understand, test, maintain and extend.

Key takeaways (1)

Architecture should make the next change easier, not merely look sophisticated

- 1 Agile still needs design** Design happens continuously and evolves with feedback.
- 2 Start with responsibilities** Separate UI, business rules, persistence and integration concerns.
- 3 Keep boundaries clear** Cohesion, low coupling and encapsulation improve changeability.
- 4 Patterns are tools** Layered architecture and MVC are useful starting points; modern patterns add trade-offs.
- 5 Quality drives architecture** Security, testability, scalability and maintainability shape decisions.
- 6 Prefer simple evolution** A modular starting point can evolve when evidence justifies more complexity.

Good architecture supports working software today and safe change tomorrow.

Sprint 1 Assessments



Sprint 1: what's due and when

- **Demo (10%):** live in your Week 4 lab (7–9 October); recording due Sunday 11 October, 9 pm
- **Team Report (15%):** Sunday 11 October, 9 pm, one submission per team via Turnitin
- **Peer Evaluation (1%):** opens Wednesday of Week 4, closes Sunday 11 October, 9 pm
- **Individual Report (4%):** Sunday 18 October, 9 pm

Sprint 1 is 30% of the course. Full details on the “Project Sprint 1 Guidelines” page in Moodle.

The Demo in next week's lab

- **10–12 minutes**, then questions from your tutor and the other teams
- **Every member attends and presents a part**; absence without special consideration is 0 for that student
- **Show the Shared Expense Splitter features you built**, running live on your own machine
- **Show how the sprint was run**: backlog, issue board, issues, branches, merge requests
- **Record the session** and submit the MP4 to Moodle (*used for cross-marking and special consideration*)

Have the app running and your GitLab tabs open before your slot starts

Team Report, Peer Evaluation and Individual Report

- **Team Report**, about 15 pages, five parts:
 - Scrum roles, role allocation and meeting minutes (3%)
 - Sprint planning: user stories, acceptance criteria, points, sprint goal (4%)
 - Sprint tracking: issue board, meetings, Git workflow (3%)
 - Velocity: points committed versus completed, and what it tells you (2%)
 - Retrospective: what went well, what did not, things to try in Sprint 2 (3%)
- **Peer evaluation** is 1% for submitting on time (*there are no late submissions*)
- **Individual report (4%, W5)**: your own contribution, evidence from issues and merge requests

References

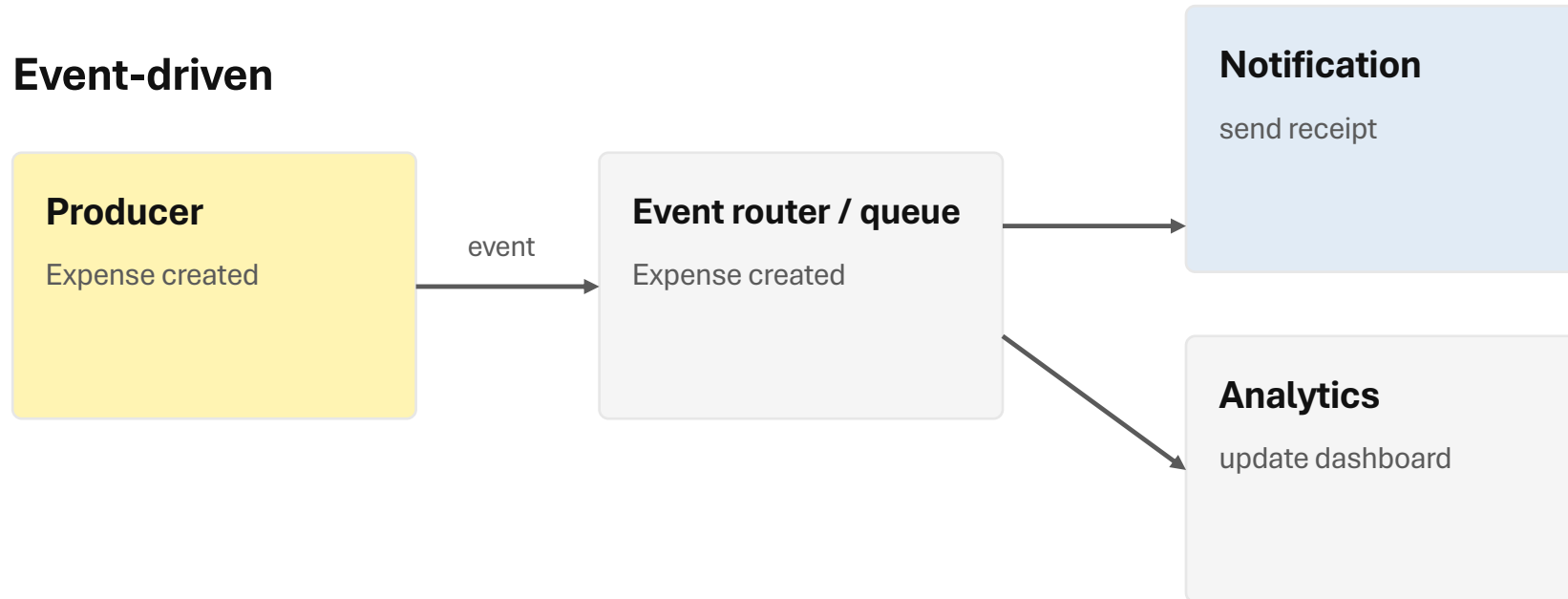
- Agile Manifesto. <https://agilemanifesto.org/>
- Schwaber, K. and Sutherland, J. The Scrum Guide, 2020. <https://scrumguides.org/scrum-guide.html>
- MDN Web Docs. Client-server overview. https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Server-side/First_steps/Client-Server_overview
- MDN Web Docs. HTTP. <https://developer.mozilla.org/en-US/docs/Web/HTTP>
- C4 model. Introduction and abstractions. <https://c4model.com/introduction> and <https://c4model.com/abstractions>
- Pallets Projects. Flask documentation: application lifecycle and project layout. <https://flask.palletsprojects.com/en/stable/lifecycle/>
- Martin Fowler. Microservices. <https://martinfowler.com/articles/microservices.html>
- AWS. Event-driven architecture. <https://aws.amazon.com/event-driven-architecture/>

Thank you

Questions and discussion

Event-driven and serverless patterns

Event-driven



Serverless idea

Run small functions or managed services on demand rather than managing long-running servers.

Useful for event-triggered tasks, APIs and variable workloads.

- Useful when work is asynchronous, bursty or naturally triggered by events
- Trade-off: asynchronous systems can be harder to trace, test and reason about.

References: AWS event-driven architecture and serverless documentation

Design for non-functional properties

Architecture is where many quality trade-offs become visible

Quality	Design question	Typical design responses
Security	Where are trust boundaries? Who can access what?	Server-side authorisation, least privilege, secure defaults
Scalability	What may grow independently?	Stateless APIs, caching, queues, selective decomposition
Performance	Where is time spent?	Fewer network calls, efficient queries, caching
Reliability	What happens when dependencies fail?	Timeouts, retries, graceful failure, observability
Maintainability	How local is a change?	Clear modules, low coupling, automated tests
Deployability	Can changes move safely?	Small increments, configuration, CI/CD-friendly boundaries

How do we choose an architecture?

Use constraints and evidence, not fashion

Product

What are the highest-value user journeys?

Team

What can this team understand, test and operate?

Change

Which parts may evolve independently?

Quality

Which non-functional properties matter now?

Operations

How much deployment and monitoring complexity can we support?

Evidence

What have we learned from previous increments?

For a small Agile web project, start with the simplest architecture that preserves clear boundaries.